1. Exam Task

The following task is part of the final project which has to be submitted until August 20, 2025. Details on how to submit will be provided later.

(1) Contraction Hierarchies (CH)

Implement CH. This includes the following vital steps:

- On ILIAS, we provide a road network data set together with a description of the respective file format. Write a suitable parser and store the data in a proper graph data structure.
- Implement Dijkstra's algorithm such that it works with your graph data structure. This algorithm will serve as a baseline and as an important ingredient in the CH preprocessing. For a given pair of source and target node IDs it should return the shortest path distance and also the shortest path itself (as a sequence of edges).
- Implement the CH preprocessing algorithm. Use the *edge difference* (ED) to decide which node to contract next. Ties can be broken arbitrarily and it is allowed to contract multiple nodes with equal ED in one round as long as they are independent. The algorithm should return a new graph, the CH-graph, which consists of the original graph plus all inserted shortcut edges. Additionally, shortcut edges should have references to the two edges that are bridged by it (for path unpacking) and the nodes should remember their rank in the contraction order.
- Implement a bidirectional CH-Dijkstra that runs on the CH-graph and only relaxes upwards edges. It should return the shortest path distance, the shortest path computed by the CH-Dijkstra (possibly containing shortcuts) and also the shortest path. Note that the latter requires to unpack shortcut edges until the path consists solely of original edges.

For more detail on the respective algorithms, consult the lecture slides or have a look at the original publications on CH: the conference paper or the journal version. Conduct the following experiments and evaluations:

- Run the CH preprocessing algorithm on all road networks in the provided data set. Measure for each graph the running time and the number of shortcuts that have been inserted. Create a plot or table with the respective results. *Hint: If the number of inserted shortcuts is significantly larger than the number of original edges, there might be something wrong.*
- On each road network, select at least 100 random source-target node pairs. Run Dijkstra on the original graph and the CH-Dijkstra on the CH-graph for each query node pair to produce the shortest path distance and the shortest path itself. Check whether the shortest path distances computed by Dijkstra and CH-Dijkstra are always identical for the same node pair. Measure the average running time of the two algorithms on each road network and also the average number of nodes popped from the respective priority queues. Create a plot or table with the respective results.
- Use a visualization tool of your choice to plot the osm5 road network together with the shortest path produced by Dijkstra (for a not too short example query), the shortest path produced by CH-Dijkstra (with shortcuts) and the unpacked shortest path (which might not necessarily coincide with the Dijkstra path in case the shortest path is ambiguous. Make sure that one can see all paths in any case). Produce four different such pictures. Also plot the CH-graph of osm5 (once).